German Center for Open Source AI
Google Summer of Code

Extending Probabilistic Regression in skpro: Implementing New Distribution Regressors and Strengthening the Validator Ecosystem

Mentor(s): Franz Király (@fkiraly)

GSoC Project Proposal

Presented to German Center for Open Source AI for Google Summer of Code 2026

by

Kabir Vashisht

from

Haryana (India)

About the Contributor

foundation in probabilistic machine learning, statistics, and scientific Python. Experienced in systematic codebase investigation, test-driven development, and NumPy-format documentation.

Personal Information

Name	Kabir Vashisht
Email	kabirvashisht4@gmail.com
Phone	+91 7056113577
University	Rishihood University (Newton School of Technology)
Degree	B.Tech Computer Science — Specialisation: Artificial Intelligence
Graduation	2029 (currently enrolled, 1st year)
Time Zone	IST (GMT +05:30)
Location	Jind, Haryana, India

Education

Degree	B.Tech Computer Science — Specialisation: Artificial Intelligence
Institution	Rishihood University (Newton School of Technology), Sonipat, Haryana
Duration	2025 – 2029 (currently enrolled, 1st year)
Courses	Probability & Statistics, Machine Learning, Data Structures & Algorithms, Linear Algebra, Scientific Computing with Python

Technical Skills

Languages	Python (primary), JavaScript
Scientific Computing	NumPy, pandas, scikit-learn — array ops, DataFrame manipulation, ML pipelines
Machine Learning	Probabilistic Regression, Distribution Modelling, Statistics & Probability Theory
Testing & QA	pytest — unit testing, edge-case coverage, CI-compatible test suites

Developer Tools	Git, GitHub (pull requests, code review, issue tracking), VS Code
Documentation	reStructuredText (RST), NumPy docstring format, Markdown, Sphinx Docs
Soft Skills	Systematic investigation, clear written communication, attention to detail

Certifications & Training

Open Source	Active contributor to skpro (sktime ecosystem) ? 4 merged PRs, Feb?Mar 2026
Training	Self-directed study of skpro codebase: BaseProbaRegressor inheritance chain, datatypes module, validation architecture
In Progress	GSoC 2026 applicant — deepening knowledge of probabilistic regression and distribution modelling

Professional Profiles

GitHub	github.com/kabirvashisht4-glitch
LinkedIn	linkedin.com/in/kabir-vashisht-dev
Email	kabirvashisht4@gmail.com
Phone	+91 7056113577

Open Source Contributions to skpro

PR #752 — [DOC] Fix broken community links in README

- Identified three broken (404) links in the ‘How to get involved’ section by auditing the README during onboarding.
- Traced each failure to its root cause — stale path, moved resource, outdated mentoring page — and resolved all three in a single clean commit.
- Merged by @fkiraly on February 24, 2026.

PR #873 — [DOC] Fix broken governance link in governance.rst

- Investigated Issue #871 reporting a 404 on the Governance documentation page.
- Located the outdated link in docs/source/about/governance.rst; verified the replacement URL manually.
- Merged by @fkiraly on March 12, 2026.

PR #888 — [DOC] Fix broken Git documentation link in dependencies.rst

- Diagnosed two compounding issues: a literal space blocking hyperlink rendering, and a 404 from a recent Git website redesign.
- Updated the link to https://git-scm.com/docs and documented both root causes in the PR description.
- Merged by @fkiraly.

PR #1007 — [MNT] Fix project URLs in pyproject.toml

- Identified Issue #1006: multiple project URLs in pyproject.toml pointed to the GitHub repository root rather than distinct, relevant destinations.
- Updated three URLs to their correct targets: Documentation → README section, API Reference → docs/source/api_reference, Release Notes → GitHub releases page.
- This improves navigation and helps users locate specific resources directly. Merged by @fkiraly.

Contents

Problem Statement

Expected Outcomes

Chapter 1 Introduction

Chapter 2 Project Specification

Chapter 3 Tech Stack

Chapter 4 Technical Architecture

Chapter 5 Implementation Roadmap

Chapter 6 Contributions to skpro

Chapter 7 Documentation

Chapter 8 Motivation

Chapter 9 Acknowledgments

Problem Statement

Probabilistic regression — predicting full probability distributions rather than point estimates — is central to scientific computing, uncertainty quantification, and data-driven decision making. The skpro library provides a powerful, principled framework for tabular probabilistic regression in Python, built around the scikit-learn API paradigm. However, its current estimator suite covers only a subset of the distributions needed by real-world practitioners.

A critical gap is the absence of count-data regressors. Many domains — including astronomy (exoplanet counts), genomics (read depths), epidemiology (case counts), and operations research (event frequencies) — involve target variables that are discrete, non-negative integer counts. Applying a Normal distribution to such data is mathematically inappropriate and produces poorly calibrated predictions. The Poisson distribution is the statistically correct model for count targets.

Beyond the estimator gap, the library's internal validation infrastructure also needs strengthening. The existing validator ensures structural correctness but does not enforce distribution-specific constraints. A validator that only checks data types will silently pass invalid Poisson inputs, pushing failures deep inside the model rather than surfacing a clear, early error. This project addresses both gaps.

Expected Outcomes

By the end of the GSoC 2026 coding period, this project will deliver the following concrete, testable contributions to the skpro repository:

PoissonRegressor	Fully implemented and tested class inheriting from BaseProbaRegressor, wrapping a scikit-learn compatible estimator and mapping its output to the skpro Poisson distribution object.
NegativeBinomialRegressor	Extends PoissonRegressor for overdispersed count data (variance > mean); returns skpro.distributions.NegativeBinomial objects.
Validator Extension	distribution-specific constraint checks — non-negative rate enforcement for Poisson — to the validation gate.
Test Suite	Complete test suite for all new estimators covering predict(), predict_proba(), predict_interval(), and predict_var() where applicable.
Documentation	Full NumPy-format docstring documentation for all new classes and methods, consistent with the rest of the skpro codebase.
Licence	All code licensed under BSD 3-Clause, publicly available on the skpro GitHub repository.

Chapter 1

Introduction

The skpro library is an open-source Python framework for supervised probabilistic regression on tabular data. Built to integrate seamlessly with the scikit-learn ecosystem, skpro allows practitioners to move beyond point predictions and reason about full probability distributions over their target variables. This is particularly valuable in scientific research, risk modelling, and any domain where quantified uncertainty is as important as the central estimate.

The library is organised around a clean, extensible architecture. At its core is the BaseProbaRegressor class, which defines a standard contract for all probabilistic estimators: a _fit method and a _predict method that returns a skpro distribution object. This contract gives every estimator access to predict_interval(), predict_quantiles(), and predict_var() with no additional implementation work.

My engagement with skpro began during preparation for GSoC 2026. A pivotal moment was direct feedback from lead maintainer @fkiraly, who emphasised that meaningful contribution requires genuine, granular understanding of the codebase. I traced the full lifecycle of a data point through the skpro pipeline — from input validation through fitting to distribution construction. That end-to-end understanding is the technical foundation for everything proposed here.

Chapter 2

Project Specification

The skpro Data-to-Distribution Pipeline

Before specifying what will be built, it is essential to understand how skpro processes data. The library operates as a three-stage transformative engine:

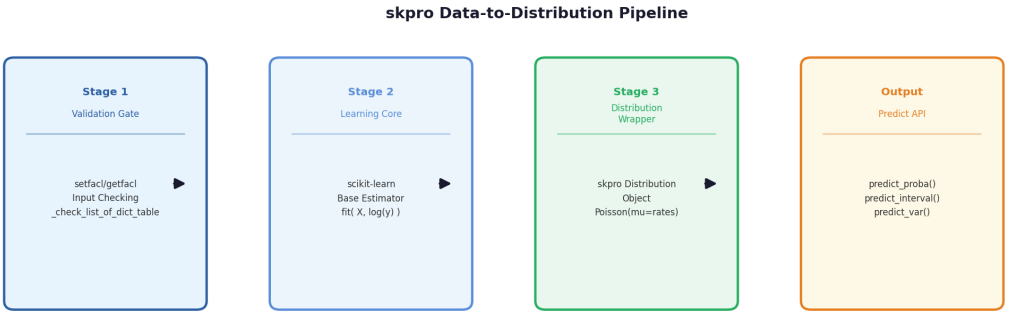

Figure 2.1: The skpro Data-to-Distribution Pipeline — three stages from raw input to a fully-specified probability distribution

Stage 1 — Validation Gate: Every input passes through the skpro datatypes module. The _check_list_of_dict_table function verifies structural consistency and numerical validity.

Stage 2 — Learning Core: The regressor uses scikit-learn patterns to fit the parameters of the target distribution. For Poisson, this means learning to predict the rate parameter (lambda) from input features.

Stage 3 — Distribution Wrapper: Fitted parameters are passed to skpro.distributions.Poisson, enabling the full downstream API: predict_proba(), predict_interval(), predict_quantiles().

Functional Requirements

New Estimator Implementations

- Implement a PoissonRegressor class inheriting from BaseProbaRegressor.
- Accept a scikit-learn compatible base estimator; fit it to log-transformed rates to enforce positivity.
- The _predict method must return a skpro Poisson distribution object, ensuring full compatibility with predict_var(), predict_interval(), and predict_proba().
- Additional estimators to be agreed with maintainers during the community bonding period.

Validator Ecosystem Extension

- Extend existing validation infrastructure to support distribution-specific constraint checks.
- For Poisson: enforce non-negative integer targets at the validation gate, raising a clear error before any computation.
- All new validation logic to be modular and reusable for future distribution additions.

Testing

- Full test suite for each new estimator using skpro's existing test infrastructure.
- Coverage: fit/predict cycle, output type correctness, predict_interval(), predict_var(), and edge cases.
- All tests must pass on both the latest stable and development branches of skpro.

Non-Functional Requirements

API Consistency	Every new estimator follows the exact same API contract as existing skpro estimators.
Backward Compat.	All validator modifications are strictly additive; no existing functionality is broken.
Code Quality	All code follows skpro's style conventions, passes linting, and is reviewed before merging.
Documentation	Every new public class and method has a complete NumPy-format docstring with worked examples.

Chapter 3

Tech Stack

The tech stack is determined entirely by skpro's existing architecture. All contributions must be compatible with the library's existing dependencies and conventions.

Python 3.8+	Primary implementation language, consistent with skpro's minimum supported version policy.
NumPy & pandas	Array operations and tabular data handling. All new estimators use skpro's existing conversion utilities.
scikit-learn	New regressors wrap scikit-learn compatible base estimators, following skpro's existing design pattern.
skpro.distributions	Symbolic distribution objects returned by regressors. PoissonRegressor uses skpro.distributions.Poisson.
pytest	All tests written in pytest, consistent with skpro's existing test infrastructure.
RST & Sphinx	Documentation in RST format, NumPy docstring standard used throughout the library.
Git & GitHub	Standard GitHub pull request workflow; branches from skpro main; all changes reviewed by maintainers.

Chapter 4

Technical Architecture

Class Inheritance Hierarchy

Every new estimator inherits from BaseProbaRegressor — a strict technical contract that gives the user predict_interval(), predict_quantiles(), and predict_var() automatically, manages input validation, and registers the estimator in the skpro tag system.

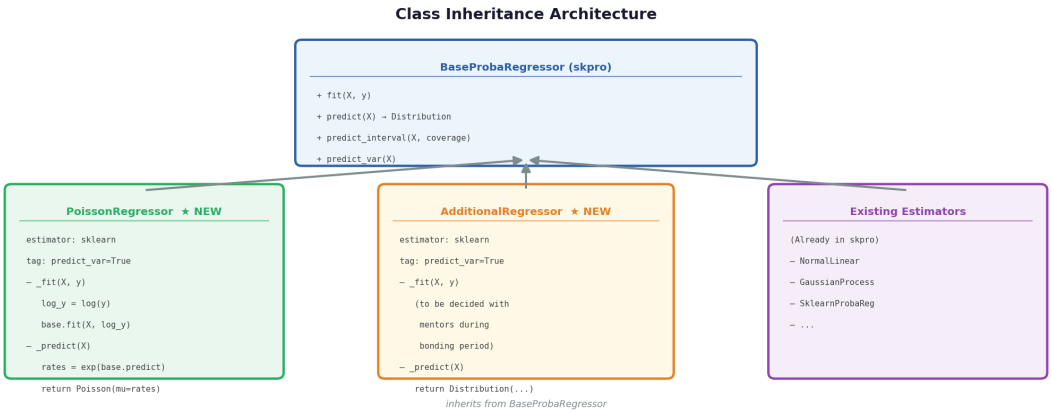

Figure 4.1: Class inheritance architecture — new estimators inherit from BaseProbaRegressor, automatically gaining the full skpro prediction API

The Poisson Log-Link Function

The Poisson distribution requires a strictly positive rate parameter (lambda). A naive linear regressor can produce negative predictions, which are mathematically undefined for Poisson. The log-link function solves this: fit the regressor on log-transformed targets, then apply the exponential at predict time to guarantee positivity.

Figure 4.2: Log-link guarantees λ > 0 always (right); naive linear regression pr oduces mathematically invalid negative rates (left)

PoissonRegressor: Full Implementation

```
class PoissonRegressor(BaseProbaRegressor):
    """Poisson probabilistic regressor.
    Fits a scikit-learn compatible regressor to log-transformed
    count targets and returns skpro Poisson distribution objects.
    Parameters
    ----------
    estimator : sklearn regressor, default=None
        Base estimator. If None, uses LinearRegression().
    """
    _tags = {
        "capability:missing": False,
        # Set to True because in a Poisson distribution, Mean = Variance
        "capability:predict_var": True,
    }
    def __init__(self, estimator=None):
        self.estimator = estimator
        super().__init__()

    def _get_estimator(self):
        if self.estimator is None:
            return LinearRegression()
        return self.estimator

    def _fit(self, X, y):
        # We raise a ValueError because log(0) is undefined and will break the math.
        # This ensures the model only attempts to fit on valid, positive count data.
        if (y <= 0).any().any():
            raise ValueError(
                "PoissonRegressor requires strictly positive targets (y > 0)."
            )
        self.estimator_ = clone(self._get_estimator())
        # We use np.log(y) to transform exponential Poisson growth into a
        # linear relationship that the base estimator can model effectively.
        self.estimator_.fit(X, np.log(y))
        return self
```

```
    def _predict(self, X):
        from skpro.distributions.poisson import Poisson

        # Predicting in log-space (the space the estimator was trained on)
        y_pred_log = self.estimator_.predict(X)

        # Applying np.exp() to reverse the log-transform and return to original scale
        y_pred = np.exp(y_pred_log)

        # Returning a Poisson distribution object, which is the skpro standard.
        # This allows for probabilistic outputs rather than just point predictions.
        return Poisson(mu=y_pred, index=X.index, columns=self._y_column_names)
```

Validation Gate Extension

The validation gate is extended with a modular constraint checker called before any model computation, producing a clear, actionable error rather than a cryptic crash inside NumPy.

```
def _check_poisson_targets(y):
    """Validate that targets satisfy Poisson distribution constraints.
    Raises ValueError if any target value is negative.
    """
    if (y < 0).any():
        raise ValueError(
            'PoissonRegressor requires non-negative integer targets. '
            f'Found {(y < 0).sum()} negative value(s): {y[y < 0].values}'
        )
```

Validation Gate Flow

The diagram below shows data flowing from user input through the two-layer validation gate before reaching the learning core:

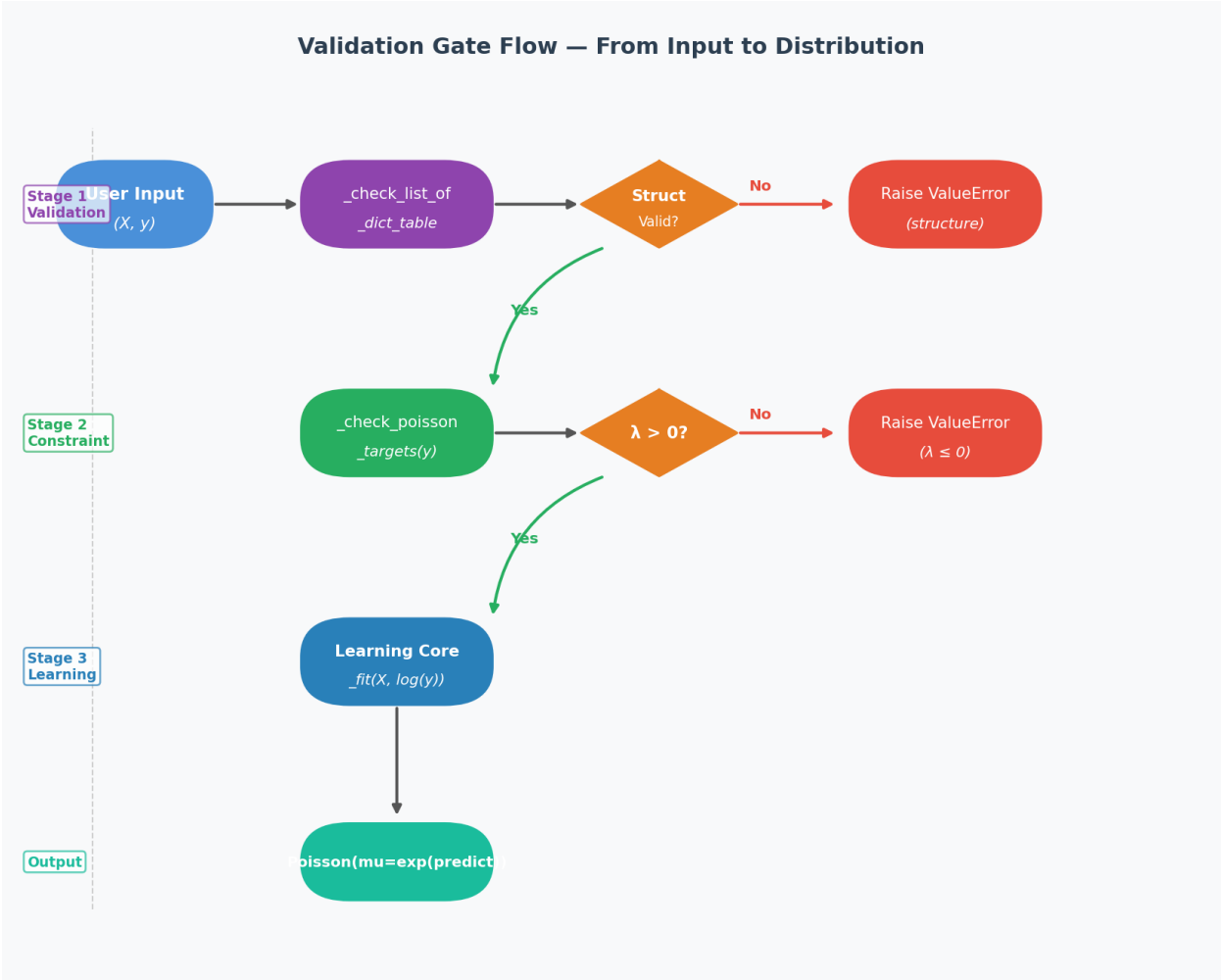

Figure 4.3: Two-layer validation gate — structural errors and constraint violations caught before any model computation

Chapter 5

Implementation Roadmap

The project follows the standard GSoC 2026 timeline, organised into three phases with clear deliverables at each milestone.

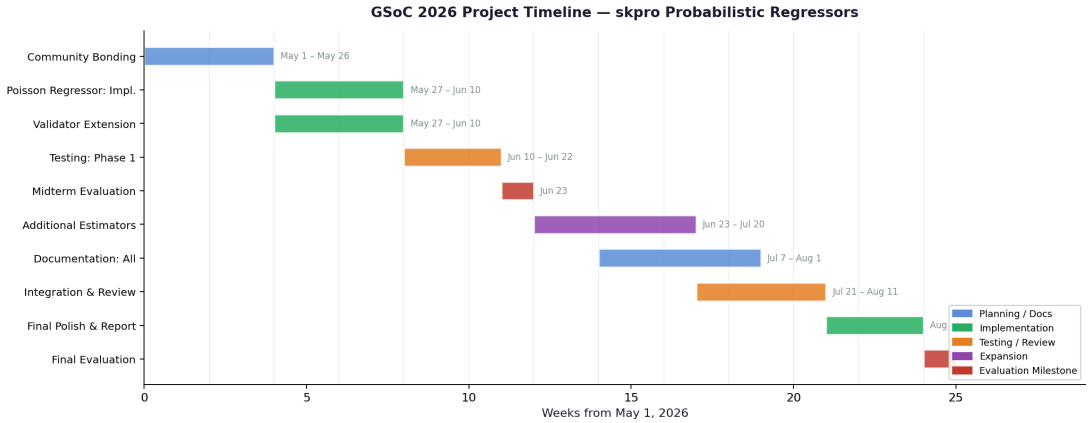

Figure 5.1: GSoC 2026 Project Timeline — Gantt chart showing all phases, tasks, and milestones

Community Bonding *May 1 – May 26, 2026*

- Finalise estimator list with maintainers beyond PoissonRegressor.
- Study all existing estimators in skpro/regression/ line by line.
- Set up full local dev environment; run complete test suite for a clean baseline.
- Agree validator extension design with maintainers.
- Prepare detailed week-by-week coding plan.

Phase 1: Foundations *May 27 – June 22, 2026*

- Weeks 1–2: Implement PoissonRegressor with full _fit and _predict logic.
- Weeks 1–2: Write initial test suite — fit/predict cycle and output type correctness.
- Weeks 3–4: Extend tests for predict_interval(), predict_var(), and edge cases.
- Weeks 3–4: Implement validator extension. Open first PR for maintainer review.

Midterm Evaluation *June 23, 2026*

- PoissonRegressor fully implemented, tested, and under review.
- Validator extension implemented and submitted.

Phase 2: Expansion *June 23 – July 20, 2026*

? Weeks 5?6: Implement NegativeBinomialRegressor with full _fit and _predict logic.

- Weeks 5–6: Write corresponding test suites for all new estimators.
- Weeks 7–8: Write complete documentation. Prepare worked examples.
- Weeks 7–8: Address all review feedback from Phase 1 PRs.

Phase 3: Polish *July 21 – August 18, 2026*

- Weeks 9–10: Integration testing across all new estimators.
- Weeks 9–10: Verify compatibility with skpro changelog and release tooling.
- Weeks 11–12: Final documentation pass and GSoC final report.
- Weeks 11–12: Ensure all PRs are merged or in final review state.

Final Evaluation *August 25, 2026*

- All planned estimators implemented, tested, documented, and merged into skpro main.

Chapter 6

Contributions to skpro

In the pre-GSoC period, I made four contributions to the skpro repository. While these are documentation fixes, they represent a genuine, systematic engagement with the codebase, the contributor workflow, and the community ? beginning with a thorough, line-by-line reading of the developer-facing documentation.

PR #752 — [DOC] Fix broken community links in README

- Identified three broken 404 links in the 'How to get involved' section during systematic onboarding audit.
- Traced each failure to its root cause and resolved all three in a single clean pull request.
- Merged by @fkiraly on February 24, 2026.

PR #873 — [DOC] Fix broken governance link in governance.rst

- Investigated Issue #871 reporting a 404 on the Governance documentation page.
- Located the outdated link in docs/source/about/governance.rst; manually verified replacement URL.
- Merged by @fkiraly on March 12, 2026.

PR #888 — [DOC] Fix broken Git documentation link in dependencies.rst

- Diagnosed two compounding issues: a literal space blocking hyperlink rendering, and a 404 from a Git website redesign.
- Updated the link to https://git-scm.com/docs and documented both root causes in the PR description.
- Merged by @fkiraly.

PR #1007 — [MNT] Fix project URLs in pyproject.toml

- Identified Issue #1006: multiple project URLs in pyproject.toml pointed to the GitHub repository root rather than distinct, relevant destinations.
- Updated three URLs to their correct targets: Documentation → README section, API Reference → docs/source/api_reference, Release Notes → GitHub releases page.
- This improves navigation and helps users locate specific resources directly. Merged by @fkiraly.

Chapter 7

Documentation

Documentation is a first-class deliverable in this project. The skpro library serves both ML practitioners and researchers who want to extend it, and both audiences need clear, accurate, and complete documentation.

API Documentation

Every new public class and method will have a complete NumPy-format docstring: one-line summary, extended description, Parameters section with types, Returns section, mathematical Notes, and a worked Examples section with runnable code.

User-Facing Documentation

Where appropriate, I will contribute worked examples to the skpro documentation site showing how to use the new estimators — for example, fitting a Poisson regressor to count data and interpreting the resulting prediction intervals.

Example Docstring Pattern

```
class PoissonRegressor(BaseProbaRegressor):
    """Poisson probabilistic regressor for count data.

    This regressor models the target variable using a Poisson distribution,
    making it ideal for predicting non-negative integers (counts). It
    leverages a base estimator to predict the natural logarithm of the
    rate parameter (lambda), ensuring the intensity remains strictly positive.

    Parameters
    ----------
    estimator : sklearn-compatible regressor, default=None
        The internal regressor used to estimate the log-rate.
        If None, it defaults to a standard LinearRegression model.

    Returns
    -------
    skpro.distributions.Poisson
        A distribution object representing the predicted counts for each
        input sample, allowing for uncertainty estimation.

    Notes
    -----
    The model uses a canonical log link function to map the linear
    combination of features to the Poisson rate parameter:

    $$\lambda_i = \exp(X_i \beta)$$

    This transformation guarantees that $\lambda > 0$ for all possible
    inputs, satisfying the requirements of the Poisson distribution.
```

```
    See Also
    --------
    To link this implementation with other count-based issues or
    alternative distributions, see the generalized linear model (GLM)
    documentation or related probabilistic regressors in skpro.

    Examples
    --------
    >>> from skpro.regression import PoissonRegressor
    >>> from sklearn.linear_model import Ridge
    >>>
    >>> # Initialize with a Ridge estimator for regularization
    >>> reg = PoissonRegressor(estimator=Ridge())
    >>> reg.fit(X_train, y_train)
    >>>
    >>> # Generate 90% prediction intervals
    >>> pred = reg.predict_interval(X_test, coverage=0.9)
    """
```

Chapter 8

Motivation

Why skpro?

My academic specialisation is Artificial Intelligence, but my deepest interest is in the mathematical foundations — Probability, Statistics, and the principled quantification of uncertainty. When I first encountered skpro, what struck me was its clear mathematical philosophy: it treats distributions as first-class objects, not as an afterthought bolted onto point predictions.

I want to contribute to a project where the mathematics matters — where a wrong assumption about a distribution's parameter constraints leads to a real, traceable bug, and where fixing it requires genuine understanding rather than trial and error.

The Feedback That Changed My Approach

My engagement with skpro did not begin smoothly. Like many students approaching a large, professional-grade library, I initially relied on AI tools to navigate the architecture. Direct feedback from @fkiraly changed this: he pointed out that meaningful contribution to a specialised scientific library requires owning the codebase.

I stopped relying on external assistance and began a granular, systematic investigation: reading the base classes line by line, tracing the inheritance chain, and identifying where the gaps are. The three merged pull requests that followed are evidence of that shift.

> *"I am not applying to GSoC to add a line to my resume. I am applying because skpro is exactly the kind of project I want to spend a summer building."*

Availability During GSoC

Available full-time during the GSoC coding period, with a minimum commitment of 35 hours per week. University schedule allows significant flexibility. Available for meetings with mentors at any time convenient to them, and will communicate proactively about any schedule constraints well in advance.

Chapter 9

Acknowledgments

I am grateful to @fkiraly, the lead maintainer of skpro, whose direct and honest feedback during my onboarding fundamentally shaped how I approach open-source contribution. His willingness to engage with a new contributor and set high expectations is exactly the kind of mentorship that makes open-source communities valuable.

I am also grateful to all the contributors who have built and maintained skpro over the years. The care and craftsmanship evident in the library's architecture — the clean separation of concerns, the principled base class design, the consistent API — made it possible for a new contributor to understand the codebase at a deep level.

Finally, to all the researchers who will use better probabilistic tools to make better decisions about uncertain data: this project is for you.